

Wydział Matematyki
i Nauk Informacyjnych

POLITECHNIKA WARSZAWSKA

Teoria Algorytmów i Obliczeń

Projekt

Problem podziału na k podzbiorów

Autorzy
Maksymilian Majewski
Michał Napieralski
Łukasz Przybylski
Filip Sewastianowicz

8 listopada 2026

Spis treści

1 Wstęp **3**
1. Opis problemu . . . 3

2 Algorytm Karmarkara–Karpa **4**
1. Ogólny opis . . . 4
2. Pojęcia i kroki algorytmu . . . 4
3. Dowód poprawności . . . 5
4. Jakość rozwiązania . . . 6
5. Złożoność obliczeniowa . . . 7

3 Algorytm zachłanny (Greedy Number Partitioning) **9**
1. Ogólny opis . . . 9
2. Pojęcia i kroki algorytmu . . . 9
3. Dowód poprawności . . . 10
4. Jakość rozwiązania . . . 11
5. Złożoność obliczeniowa . . . 12

4 Programowanie liniowe całkowitoliczbowe (LP) **13**
1. Ogólny opis . . . 13
2. Pojęcia i kroki algorytmu . . . 13
3. Dowód poprawności . . . 15
4. Jakość rozwiązania . . . 16
5. Złożoność obliczeniowa . . . 17

5 Pruning w modelu LP **19**
1. Problem i zakres optymalizacji . . . 19
2. Rozwiązanie początkowe i ograniczenie rozpiętości . . . 19
3. Granice sum grup wynikające z rozpiętości . . . 19
4. Redukcja symetrii przez porządek sum . . . 19
5. Co gwarantuje obecna implementacja . . . 20

6 Orkiestrator: działanie, złożoność i poprawność **21**
Cel i oznaczenia . . . 21
CV i wybór ścieżki . . . 21
Przebieg algorytmu . . . 21
Złożoność obliczeniowa . . . 21

7 Architektura implementacji **23**
Organizacja programu . . . 23
Podział odpowiedzialności . . . 23
Reprezentacja danych i współpraca modułów . . . 24

8 Wybór progu CV orkiestratora **25**
1. Problem . . . 25
2. Kluczowa obserwacja . . . 25
3. Funkcja celu . . . 25
4. Procedura . . . 26
5. Uruchomienie . . . 26
6. Wyniki . . . 26

9 Porównanie orkiestratora z algorytmem Karmarkara–Karpa **28**
1. Cel . . . 28
2. Metoda . . . 28
3. Uruchomienie . . . 29
4. Wyniki . . . 29
5. Podsumowanie . . . 32

10 Wnioski **33**
1. Zakres zrealizowanych prac . . . 33
2. Najważniejsze wyniki . . . 33
3. Ograniczenia . . . 34
4. Perspektywy i dalsze prace . . . 34

Bibliografia **35**

Rozdział 1

Wstęp

1. Opis problemu

Przedmiotem projektu jest problem podziału liczb [11]. Dany jest zbiór liczb naturalnych $S = \{a_1, \dots, a_n\}$ oraz liczba $k \geq 1$. Należy podzielić S na k rozłącznych podzbiorów $P_1, \dots, P_k$ których suma daje cały zbiór S, w taki sposób, aby sumy liczb w podzbiorach były możliwie zbliżone.

Jako miarę jakości podziału przyjmujemy rozpiętość, czyli różnicę między największą a najmniejszą sumą liczb w podzbiorach:

$$D(P_1, \dots, P_k) = \max_j \sum_{a \in P_j} a \; - \; \min_j \sum_{a \in P_j} a.$$

Szukamy podziału o najmniejszej rozpiętości. Tę wartość minimalizuje model LP i według niej porównujemy algorytmy w eksperymentach. Aby porównywać zbiory o różnej skali, posługujemy się także rozpiętością względną D/Σ, gdzie $\Sigma = \sum_{i=1}^{n} a_i$.

Gwarancje teoretyczne algorytmów przybliżonych podajemy, zgodnie z literaturą, względem sumy największego podzbioru, czyli podzbioru o największej sumie liczb (nie o największej liczności). Jej najmniejszą możliwą wartość oznaczamy

$$OPT = \min_{(P_1, \dots, P_k)} \max_j \sum_{a \in P_j} a.$$

Problem jest NP-trudny. Oznacza to, że nie jest znany algorytm, który dla każdego wejścia znajduje rozwiązanie optymalne w czasie wielomianowym. Dlatego w projekcie rozważamy dwa podejścia: dokładne, które zawsze znajduje optimum, ale w najgorszym przypadku działa w czasie wykładniczym, oraz aproksymacyjne, które działa w czasie wielomianowym, ale może zwrócić rozwiązanie nieoptymalne o ograniczonym błędzie.

Rozdział 2

Algorytm Karmarkara–Karpa

1. Ogólny opis

Algorytm Karmarkara–Karpa [1, 4] jest algorytmem aproksymacyjnym dla problemu podziału liczb. Algorytm Karmarkara–Karpa działa w czasie wielomianowym i zwraca rozwiązanie, które może być nieoptymalne. Jego błąd jest jednak ograniczony z góry, co wykażemy w sekcji 4.

2. Pojęcia i kroki algorytmu

2.1. Pojęcia

- **Grupa** – lista liczb ze zbioru S. Grupy będą kandydatami na docelowe podzbiory.
- **Wartość grupy** $w(G)$ – suma liczb w grupie G (dla grupy pustej $w(G) = 0$).
- **Wiersz** – k-elementowa krotka grup $R = (G_1, \dots, G_k)$.
- **Rozpiętość wiersza** – różnica między największą i najmniejszą wartością grupy w wierszu:
$$\operatorname{rozp}(R) = \max_i w(G_i) - \min_i w(G_i).$$
- **Sortowanie wiersza** – ustawienie grup w kolejności rosnących wartości. Wszystkie wiersze utrzymujemy posortowane.
- **Odwrócenie wiersza** – odwrócenie kolejności grup w wierszu.

2.2. Kroki algorytmu

Inicjalizacja. Dla każdej liczby $a_i \in S$ tworzymy wiersz

$$([\,],\ \dots,\ [\,],\ [a_i]),$$

czyli $k-1$ pustych grup i jedną grupę zawierającą a_i. Taki wiersz jest posortowany, a jego rozpiętość wynosi a_i. Otrzymujemy n wierszy.

Pętla główna. Dopóki jest więcej niż jeden wiersz:

1. Wybierz dwa wiersze o największych rozpiętościach. Wiersz o mniejszej rozpiętości oznacz jako B, a drugi jako A (przy równych rozpiętościach wybór jest dowolny).
2. Odwróć wiersz B. Teraz jego grupy są ustawione malejąco według wartości.
3. Dołącz B do A pozycyjnie: wszystkie liczby z i-tej grupy B dopisz do i-tej grupy A ($i = 1, \dots, k$). W efekcie:
   - najmniejsza grupa A łączy się z największą grupą B,
   - druga najmniejsza grupa A łączy się z drugą największą grupą B,
   - i tak dalej.

4. Posortuj wiersz A, bo wartości jego grup mogły się zmienić.
5. Usuń pusty już wiersz B.

Wynik. Po zakończeniu pętli zostaje jeden wiersz z k grupami. Każda grupa jest jednym z docelowych podzbiorów. To, że jest to poprawny podział, wykazujemy w sekcji 3.

2.3. Przykład

$S = \{8, 7, 6, 5, 4\}$, $k = 2$. Rozpiętość wiersza podajemy w nawiasie.

Krok	Wiersze
start	$([\,], [8])_{(8)},\ ([\,], [7])_{(7)},\ ([\,], [6])_{(6)},\ ([\,], [5])_{(5)},\ ([\,], [4])_{(4)}$
1	$([7], [8])_{(1)},\ ([\,], [6])_{(6)},\ ([\,], [5])_{(5)},\ ([\,], [4])_{(4)}$
2	$([5], [6])_{(1)},\ ([7], [8])_{(1)},\ ([\,], [4])_{(4)}$
3	$([4, 7], [8])_{(3)},\ ([5], [6])_{(1)}$
4	$([8, 6], [4, 7, 5])_{(2)}$

Wynik to sumy 14 i 16. Optymalny podział $\{8, 7\}, \{6, 5, 4\}$ ma sumy równe 15, więc algorytm nie zawsze znajduje optimum.

3. Dowód poprawności

Algorytm jest aproksymacyjny, więc nie dowodzimy optymalności wyniku. Dowód poprawności składa się z trzech części:

1. algorytm się zatrzymuje (sekcja 3.1),
2. zwraca poprawny podział zbioru S na k podzbiorów (sekcja 3.2),
3. wynik jest gorszy od optimum co najwyżej o ograniczoną wartość (sekcja 4).

3.1. Własność stopu

Każda iteracja pętli usuwa dokładnie jeden wiersz (B). Na początku jest n wierszy, więc pętla wykona się dokładnie $n - 1$ razy. Algorytm zawsze się zatrzymuje.

3.2. Poprawność wyniku

Niezmiennik. Przed każdą iteracją każda liczba z S należy do dokładnie jednej grupy dokładnie jednego wiersza.

- *Inicjalizacja:* każda liczba a_i znajduje się tylko w jednej grupie swojego wiersza. Niezmiennik zachodzi.
- *Zachowanie:* w iteracji liczby z grup wiersza B są przenoszone do grup wiersza A, a B jest usuwany. Żadna liczba nie zostaje zgubiona ani zduplikowana, a pozostałe wiersze się nie zmieniają. Niezmiennik jest zachowany.

Zakończenie. Zostaje jeden wiersz - z niezmiennika wynika, że jego k grup jest parami rozłącznych, a ich suma daje S. Jest to więc poprawny podział S na k podzbiorów. Gdy $n < k$, część podzbiorów jest pusta, co jest dopuszczalne.

4. Jakość rozwiązania

4.1. Lemat: rozpiętość nie rośnie

Rozważmy jedno złączenie wierszy A i B (kroki 2–3 pętli). Oznaczmy wartości grup:

- w wierszu A: $x_1 \le x_2 \le ... \le x_k$,
- w wierszu B przed odwróceniem: $y_1 \le y_2 \le ... \le y_k$.

Po złączeniu i-ta grupa ma wartość

$$z_i = x_i + y_{k+1-i}.$$

Lemat.

$$\mathrm{rozp}(A \oplus B) \le \max\big(\mathrm{rozp}(A),\ \mathrm{rozp}(B)\big).$$

Dowód. Weźmy dowolne $i < j$. Wtedy

$$z_j - z_i = \underbrace{(x_j - x_i)}_{p\ \in\ [0,\ \mathrm{rozp}(A)]} - \underbrace{(y_{k+1-i} - y_{k+1-j})}_{q\ \in\ [0,\ \mathrm{rozp}(B)]}.$$

Obie liczby p i q są nieujemne, więc

$$|p - q| \le \max(p, q) \le \max\big(\mathrm{rozp}(A), \mathrm{rozp}(B)\big).$$

Nierówność zachodzi dla każdej pary grup, a więc także dla pary z największą i najmniejszą wartością. Oznaczenia A oraz B bezstratne w ogólności.

4.2. Wniosek: ograniczenie rozpiętości wyniku

Wprowadźmy oznaczenia:

- $a_{\max} = \max_i a_i$ – największa liczba w S,
- M – największa wartość grupy w wyniku algorytmu,
- m – najmniejsza wartość grupy w wyniku algorytmu.

Każdy wiersz początkowy ma rozpiętość $a_i \le a_{\max}$. Z lematu wynika, że żadne złączenie nie tworzy wiersza o rozpiętości większej niż rozpiętości wierszy łączonych. Przez indukcję po iteracjach każdy wiersz w trakcie działania algorytmu ma rozpiętość co najwyżej $a_{\max}$. W szczególności dotyczy to wiersza końcowego:

$$M - m \le a_{\max}.$$

4.3. Twierdzenie: gwarancja jakości

Wprowadźmy jeszcze:

- $\Sigma = \sum_{i=1}^{n} a_i$ – suma wszystkich liczb,
- OPT – wartość optymalnego rozwiązania. Dla danego podziału S na k grup patrzymy na grupę o największej sumie; OPT to najmniejsza taka suma, jaką można uzyskać spośród wszystkich możliwych podziałów:

$$OPT = \min_{(P_1,\ldots,P_k)} \max_i \sum_{a \in P_i} a.$$

Twierdzenie.

$$M \leq \frac{\Sigma}{k} + \frac{k-1}{k}\, a_{\max}.$$

Dowód. Z sekcji 4.2 każda z k grup wyniku ma wartość co najmniej $M - a_{\max}$. Jedna grupa ma wartość M, a pozostałe $k-1$ grup co najmniej $M - a_{\max}$, więc

$$\Sigma \geq M + (k-1)(M - a_{\max}) = kM - (k-1)\, a_{\max}.$$

Po podzieleniu przez k i przekształceniu otrzymujemy tezę.

Dla optimum zachodzą dwa proste oszacowania:

- $OPT \geq \Sigma/k$, bo największy podzbiór ma sumę nie mniejszą od średniej,
- $OPT \geq a_{\max}$, bo największa liczba musi trafić do któregoś podzbioru.

Podstawiając je do twierdzenia, otrzymujemy:

- **błąd bezwzględny:**

$$M - OPT \leq \frac{k-1}{k}\, a_{\max},$$

- **błąd względny:**

$$M \leq \left(2 - \frac{1}{k}\right) OPT.$$

4.4. Wyniki z literatury

Oszacowanie z sekcji 4.3 jest zgrubne. W literaturze udowodniono dokładniejsze (przytaczamy je bez dowodu):

- dla $k = 2$: $M \leq \frac{7}{6}\, OPT$ [2],
- dla dowolnego k: $M \leq \left(\frac{4}{3} - \frac{1}{3k}\right) OPT$. Istnieją instancje, dla których $M/OPT \geq \frac{4}{3} - \frac{1}{3(k-1)}$ [3].

5. Złożoność obliczeniowa

5.1. Założenia implementacyjne

- Wiersze trzymamy w kopcu uporządkowanym malejąco według rozpiętości. Moduł `heapq` realizuje kopiec typu min, więc porównanie jest odwrócone tj. na szczycie kopca znajduje się wiersz o największej rozpiętości. Wybór dwóch wierszy o największych rozpiętościach to dwa wywołania operacji `heappop`.
- Wartość $w(G)$ każdej grupy przechowujemy jako liczbę aktualizowaną przy złączeniu, zamiast liczyć za każdym razem sumę listy. Dzięki temu rozpiętość wiersza liczymy w czasie $O(1)$.
- Przy złączeniu liczby z każdej grupy wiersza B są kopiowane na koniec listy odpowiadającej grupy wiersza A.

5.2. Analiza

Oznaczmy przez L łączną liczbę liczb skopiowanych przez `extend` w trakcie całego działania algorytmu.

Etap	Koszt
Utworzenie n wierszy po k grup	$O(nk)$

Etap	Koszt
Budowa kopca (n wywołań `heappush`)	$O(n \log n)$
Jedna iteracja bez kopiowania list: 2× `heappop`, odwrócenie B, złączenie k sum, sortowanie wiersza, `heappush`	$O(\log n + k \log k)$
Liczba iteracji (sekcja 3.1)	$n - 1$
Kopiowanie liczb między listami, łącznie	$O(L)$

Daje to złożoność czasową

$$O(nk + n(\log n + k \log k) + L).$$

5.3. Oszacowanie kosztu kopiowania list

W jednej iteracji `extend` kopiuje wszystkie liczby znajdujące się w wierszu B. Wiersz nie może zawierać więcej niż n liczb, a iteracji jest $n - 1$, więc

$$L \leq n(n-1) = O(n^2).$$

Jedna liczba może być kopiowana wielokrotnie, ponieważ dzieje się to w każdej iteracji, w której jej wiersz zostaje wybrany jako B.

5.4. Podsumowanie

Złożoność czasowa (pesymistyczna):

$$O(n^2 + nk \log k).$$

Dla $k = 2$ daje to $O(n^2)$.

Jest to ograniczenie górne. Dla typowych danych wiersz B zawiera zwykle niewiele liczb i rzeczywisty koszt kopiowania jest znacznie mniejszy. Bez kopiowania list złożoność wynosiłaby $O(nk + n(\log n + k \log k))$.

Złożoność pamięciowa: $O(nk)$. Każdy z n wierszy przechowuje k obiektów, a każda liczba w danej chwili znajduje się tylko w jednej liście.

Rozdział 3

Algorytm zachłanny (Greedy Number Partitioning)

1. Ogólny opis

Algorytm zachłanny (ang. *Greedy Number Partitioning*, GNP) [5, 7] jest algorytmem aproksymacyjnym dla problemu podziału liczb. Algorytm GNP działa w czasie wielomianowym i zwraca rozwiązanie, które może być nieoptymalne. Jego błąd jest jednak ograniczony z góry, co wykażemy w sekcji 4.

Idea algorytmu jest prosta: rozpatrujemy liczby od największej do najmniejszej i każdą wkładamy do tej grupy, która w danej chwili ma najmniejszą sumę.

2. Pojęcia i kroki algorytmu

2.1. Pojęcia

- **Grupa** – lista liczb ze zbioru S. Każda z k grup jest jednym z docelowych podzbiorów.
- **Wartość grupy** $w(G)$ – suma liczb w grupie G powiększona o jej sumę początkową.
- **Sumy początkowe** $s_1, \ldots, s_k$ – wartości, od których startują grupy. W podstawowym wariancie problemu wszystkie są równe 0.
- **Najmniejsza / największa grupa** – grupa o najmniejszej / największej **wartości** $w(G)$ (nie liczności).

2.2. Kroki algorytmu

Inicjalizacja.

1. Utwórz k pustych grup $G_1, \ldots, G_k$.
2. Zbuduj kopiec typu min z par (s_i, i), czyli wartości grup razem z ich indeksami.

Pętla główna. Dla każdej liczby x z S, w kolejności **nierosnącej**:

1. Zdejmij z kopca parę (w, i) o najmniejszej wartości. Przy remisie wybierana jest grupa o mniejszym indeksie.
2. Dopisz x do grupy G_i.
3. Włóż do kopca parę $(w + x, i)$, czyli zaktualizowaną wartość grupy.

Wynik. Po przetworzeniu wszystkich liczb grupy $G_1, \ldots, G_k$ są docelowymi podzbiorami. To, że tworzą poprawny podział, wykazujemy w sekcji 3.

2.3. Przykład

$S = \{8, 7, 6, 5, 4\}$, $k = 2$, sumy początkowe $s_1 = s_2 = 0$.

Liczba	Wybrana grupa	G_1	G_2	Wartości
start	–	[ ]	[ ]	0, 0
8	G_1 (remis, mniejszy indeks)	[8]	[ ]	8, 0
7	G_2	[8]	[7]	8, 7
6	G_2	[8]	[7, 6]	8, 13
5	G_1	[8, 5]	[7, 6]	13, 13
4	G_1 (remis, mniejszy indeks)	[8, 5, 4]	[7, 6]	17, 13

Wynik to sumy 17 i 13. Optymalny podział $\{8, 7\}, \{6, 5, 4\}$ ma sumy równe 15, więc algorytm nie zawsze znajduje optimum. Dla porównania algorytm Karmarkara–Karpa na tych samych danych daje sumy 16 i 14.

3. Dowód poprawności

Algorytm jest aproksymacyjny, więc nie dowodzimy optymalności wyniku. Dowód poprawności składa się z trzech części:

1. algorytm się zatrzymuje (sekcja 3.1),
2. zwraca poprawny podział zbioru S na k podzbiorów (sekcja 3.2),
3. wynik jest gorszy od optimum co najwyżej o ograniczoną wartość (sekcja 4).

3.1. Własność stopu

Pętla główna przechodzi raz po posortowanej liście n liczb, a każda iteracja wykonuje stałą liczbę operacji na kopcu. Algorytm wykonuje dokładnie n iteracji i zawsze się zatrzymuje.

3.2. Poprawność wyniku

Niezmiennik. Przed każdą iteracją:

- (a) każda już przetworzona liczba należy do dokładnie jednej grupy, a liczby nieprzetworzone nie należą do żadnej,
- (b) kopiec zawiera dokładnie k par, po jednej dla każdej grupy, i dla każdej grupy G_i para ma postać $(w(G_i), i)$.

- *Inicjalizacja:* grupy są puste, a kopiec zawiera pary (s_i, i) dla $i = 1, \ldots, k$. Niezmiennik zachodzi.
- *Zachowanie:* iteracja zdejmuje z kopca parę (w, i), dopisuje x tylko do G_i i wkłada do kopca parę $(w + x, i)$. Liczba x trafia do dokładnie jednej grupy, inne grupy się nie zmieniają, a para grupy G_i zostaje zastąpiona parą z jej nową wartością. Niezmiennik jest zachowany.

Zakończenie: po n iteracjach wszystkie liczby są przetworzone. Z części (a) każda z nich należy do dokładnie jednej grupy, więc grupy $G_1, \ldots, G_k$ są parami rozłączne, a ich suma daje S. Jest to poprawny podział S na k podzbiorów. Gdy $n < k$, część podzbiorów jest pusta, co jest dopuszczalne. Część (b) niezmiennika gwarantuje ponadto, że w każdej iteracji zdejmowana jest grupa o rzeczywiście najmniejszej wartości. Z tego korzystamy w sekcji 4.

4. Jakość rozwiązania

W tej sekcji zakładamy, że wszystkie sumy początkowe są równe 0, tak jak w podstawowym wariancie problemu.

4.1. Lemat: rozpiętość nie przekracza największej liczby

Wprowadźmy oznaczenia:

- $a_{\max} = \max_i a_i$ – największa liczba w S,
- **rozpiętość** – różnica między największą i najmniejszą wartością grupy w danej chwili.

Lemat. Po każdej iteracji rozpiętość wynosi co najwyżej $a_{\max}$.

Dowód. Indukcja po iteracjach. Na początku wszystkie wartości są równe 0, więc rozpiętość wynosi 0.

Rozważmy iterację, w której dokładamy liczbę x. Niech przed nią najmniejsza wartość grupy wynosi $l_{\min}$, a największa $l_{\max}$, przy czym z założenia indukcyjnego $l_{\max} - l_{\min} \le a_{\max}$. Liczba x trafia do grupy o wartości $l_{\min}$ (sekcja 3.2), więc:

- nowa największa wartość to $\max(l_{\max},\ l_{\min} + x)$,
- nowa najmniejsza wartość jest nie mniejsza niż $l_{\min}$, bo wartości grup mogą tylko rosnąć.

Zatem nowa rozpiętość wynosi co najwyżej

$$\max(l_{\max},\ l_{\min} + x) - l_{\min} = \max(l_{\max} - l_{\min},\ x) \le a_{\max}.$$

4.2. Wniosek: ograniczenie rozpiętości wyniku

Wprowadźmy oznaczenia:

- M – największa wartość grupy w wyniku algorytmu,
- m – najmniejsza wartość grupy w wyniku algorytmu.

Z lematu zastosowanego do ostatniej iteracji:

$$M - m \le a_{\max}.$$

4.3. Twierdzenie: gwarancja jakości

Wprowadźmy jeszcze:

- $\Sigma = \sum_{i=1}^{n} a_i$ – suma wszystkich liczb,
- OPT – wartość optymalnego rozwiązania. Dla danego podziału S na k grup patrzymy na grupę o największej sumie; OPT to najmniejsza taka suma, jaką można uzyskać spośród wszystkich możliwych podziałów:

$$OPT = \min_{(P_1,\ldots,P_k)} \max_i \sum_{a \in P_i} a.$$

Twierdzenie.

$$M \le \frac{\Sigma}{k} + \frac{k-1}{k}\, a_{\max}.$$

Dowód. Z sekcji 4.2 każda z k grup wyniku ma wartość co najmniej $M - a_{\max}$. Jedna grupa ma wartość M, a pozostałe $k-1$ grup co najmniej $M - a_{\max}$, więc

$$\Sigma \ge M + (k-1)(M - a_{\max}) = kM - (k-1)\, a_{\max}.$$

Po podzieleniu przez k i przekształceniu otrzymujemy tezę.

Dla optimum zachodzą dwa proste oszacowania:

- $OPT \geq \Sigma/k$, bo największy podzbiór ma sumę nie mniejszą od średniej,
- $OPT \geq a_{\max}$, bo największa liczba musi trafić do któregoś podzbioru.

Podstawiając je do twierdzenia, otrzymujemy:

- **błąd bezwzględny:**
$$M - OPT \leq \frac{k-1}{k}\, a_{\max},$$
- **błąd względny:**
$$M \leq \left(2 - \frac{1}{k}\right) OPT.$$

Zauważmy, że ten dowód nie korzysta z sortowania liczb. Gwarancja $2 - \frac{1}{k}$ zachodzi dla dowolnej kolejności przetwarzania. Sortowanie malejące poprawia gwarancję, co pokazuje sekcja 4.4.

4.4. Wyniki z literatury

Dla wersji z sortowaniem malejącym (tak jak w naszej implementacji) udowodniono dokładniejsze ograniczenie (przytaczamy je bez dowodu) [6]:

$$M \leq \left(\frac{4}{3} - \frac{1}{3k}\right) OPT$$

5. Złożoność obliczeniowa

5.1. Założenia implementacyjne

- Wartości grup trzymamy w kopcu typu min (`heapq`) jako pary `(suma, indeks)`. Wybór najmniejszej grupy to jedno wywołanie `heappop`.
- Grupy są zwykłymi listami, a liczbę dopisujemy na koniec (`append`).
- Przed pętlą liczby są sortowane malejąco, co tworzy nową listę.

5.2. Analiza

Etap	Koszt
Utworzenie k pustych grup	$O(k)$
Budowa kopca (`heapify`)	$O(k)$
Sortowanie n liczb	$O(n \log n)$
Jedna iteracja: `heappop`, `append`, `heappush`	$O(\log k)$
Liczba iteracji (sekcja 3.1)	n

Złożoność czasowa:
$$O(k + n \log n + n \log k).$$
W typowym przypadku $k \leq n$ upraszcza się to do $O(n \log n)$.

Złożoność pamięciowa: $O(n+k)$. Posortowana kopia wejścia i grupy zajmują łącznie $O(n)$, a kopiec i lista grup $O(k)$.

Rozdział 4

Programowanie liniowe całkowitoliczbowe (LP)

1. Ogólny opis

Podejście oparte na programowaniu liniowym zapisuje problem podziału liczb jako model matematyczny i przekazuje go do gotowego solvera. Dany jest zbiór liczb naturalnych $S = \{a_1, \dots, a_n\}$ oraz liczba $k \geq 1$. Szukamy podziału S na k podzbiorów, których sumy są możliwie zbliżone.

Mimo skróconej nazwy LP nie jest to zwykły program liniowy. Zmienne przydziału są binarne, więc jest to program całkowitoliczbowy (ang. *Mixed-Integer Linear Programming*, MILP). Implementacja w `src/algorithm_lp.py` buduje model w bibliotece PuLP [10], a rozwiązuje go solver HiGHS [9] metodą podziału i ograniczeń.

Sam model jest dokładny: jego optimum to podział o najmniejszej rozpiętości. Implementacja przerywa jednak obliczenia po jednej sekundzie, więc w praktyce może zwrócić rozwiązanie nieoptymalne albo nie zwrócić żadnego (sekcja 4.3).

2. Pojęcia i kroki algorytmu

2.1. Pojęcia

- **Grupa** P_j – jeden z k docelowych podzbiorów.
- **Suma grupy** $w(P_j)$ – suma liczb w grupie P_j (dla grupy pustej $w(P_j) = 0$).
- **Zmienna przydziału** $x_{ij} \in \{0, 1\}$ – równa 1 dokładnie wtedy, gdy liczba a_i należy do grupy P_j.
- **Zmienne pomocnicze** $L, l \geq 0$ – ciągłe zmienne ograniczające sumy grup z góry i z dołu (w kodzie `max_sum` i `min_sum`).
- **Rozpiętość podziału** – różnica między największą i najmniejszą sumą grupy:

$$\mathrm{rozp}(P) = \max_j w(P_j) - \min_j w(P_j).$$

- **Relaksacja** – ten sam model, w którym warunek $x_{ij} \in \{0, 1\}$ zastępujemy warunkiem $x_{ij} \in [0, 1]$. Jest to zwykły program liniowy, rozwiązywalny w czasie wielomianowym.

2.2. Model

$$
\begin{aligned}
\min \quad & L - l \\
\text{przy warunkach} \quad & \sum_{j=1}^{k} x_{ij} = 1 && i = 1, \ldots, n, \\
& L \geq \sum_{i=1}^{n} a_i x_{ij} && j = 1, \ldots, k, \\
& l \leq \sum_{i=1}^{n} a_i x_{ij} && j = 1, \ldots, k, \\
& x_{ij} \in \{0, 1\}, \quad L, l \geq 0.
\end{aligned}
$$

Pierwsza rodzina warunków mówi, że każda liczba trafia do dokładnie jednej grupy. Dwie kolejne wiążą L i l z sumami grup. Model ma $nk + 2$ zmiennych i $n + 2k$ warunków.

2.3. Kroki algorytmu

Budowa modelu.

1. Utwórz nk zmiennych binarnych x_{ij}.
2. Dodaj n warunków przydziału liczb do grup.
3. Dla każdej grupy zbuduj wyrażenie liniowe opisujące jej sumę.
4. Utwórz zmienne L i l, a następnie dodaj dla każdej grupy warunki ograniczające.
5. Ustaw funkcję celu $L - l$.

Rozwiązanie. Uruchom HiGHS z limitem czasu 1 s. Solver opiera się na metodzie podziału i ograniczeń [8], której schemat wygląda następująco:

1. Rozwiąż relaksację bieżącego podproblemu. Jej wartość jest dolnym ograniczeniem wszystkich rozwiązań tego podproblemu.
2. Jeśli ograniczenie nie jest lepsze od najlepszego znanego rozwiązania, odrzuć podproblem.
3. Jeśli wszystkie x_{ij} są całkowite, zapamiętaj rozwiązanie, gdy jest lepsze od dotychczasowego.
4. W przeciwnym razie wybierz zmienną ułamkową i podziel podproblem na dwa: z $x_{ij} = 0$ oraz z $x_{ij} = 1$.

Jest to schemat ogólny, a nie dokładny opis implementacji HiGHS. Rzeczywisty solver dodatkowo upraszcza model, stosuje heurystyki i dodaje płaszczyzny tnące.

Odczyt wyniku.

1. Jeśli status solvera jest inny niż `Optimal` lub `Not Solved`, zgłoś błąd.
2. Dla każdej zmiennej x_{ij} o wartości większej niż 0,5 dopisz a_i do grupy P_j. Porównanie z 0,5 jest potrzebne, bo solver zwraca wartości zmiennoprzecinkowe.
3. Porównaj multizbiór liczb w grupach z wejściem. Jeśli się różnią, zgłoś błąd.

2.4. Przykład

$S = \{8, 7, 6, 5, 4\}$, $k = 2$. Model ma $2 \cdot 5 + 2 = 12$ zmiennych i $5 + 2 \cdot 2 = 9$ warunków:

Warunek	Postać
Przydział liczby a_i	$x_{i1} + x_{i2} = 1$, dla $i = 1, \ldots, 5$

Warunek	Postać
Górne ograniczenie	$L \geq 8x_{1j} + 7x_{2j} + 6x_{3j} + 5x_{4j} + 4x_{5j}$, dla $j = 1, 2$
Dolne ograniczenie	$l \leq 8x_{1j} + 7x_{2j} + 6x_{3j} + 5x_{4j} + 4x_{5j}$, dla $j = 1, 2$

Implementacja zwraca podział $\{6, 5, 4\}, \{8, 7\}$ o sumach równych 15, czyli $L = l = 15$ i wartość funkcji celu 0. Jest to optimum. Na tych samych danych algorytm Karmarkara–Karpa daje sumy 16 i 14, a algorytm zachłanny 17 i 13.

3. Dowód poprawności

Dowód poprawności składa się z trzech części:

1. algorytm się zatrzymuje (sekcja 3.1),
2. rozwiązania modelu odpowiadają podziałom zbioru S (sekcja 3.2),
3. optimum modelu to podział o najmniejszej rozpiętości (sekcja 3.3).

Sekcje 3.2 i 3.3 dotyczą modelu. Własności wyniku przerwanego przez limit czasu omawiamy w sekcji 4.3.

3.1. Własność stopu

Budowa modelu i odczyt wyniku to skończone pętle po n liczbach i k grupach. Drzewo podziału i ograniczeń jest skończone, bo każdy podział ustala wartość jednej z nk zmiennych binarnych. Dodatkowo limit czasu przerywa przeszukiwanie po jednej sekundzie.

3.2. Rozwiązania modelu a podziały

Lemat. Przypisania x spełniające warunki modelu odpowiadają wzajemnie jednoznacznie podziałom S na k grup, przy czym grupy mogą być puste.

Dowód. Niech x spełnia warunki. Ponieważ $x_{ij} \in \{0, 1\}$ oraz $\sum_j x_{ij} = 1$, dla każdego i istnieje dokładnie jedno j takie, że $x_{ij} = 1$. Liczba a_i należy więc do dokładnie jednej grupy. Odwrotnie, każdy podział wyznacza dokładnie jedno takie przypisanie x. Dla dowolnego x warunki na L i l można spełnić, np. przyjmując $L = \sum_i a_i$ oraz $l = 0$. Gdy $n < k$, część grup jest pusta, co jest dopuszczalne.

3.3. Funkcja celu a rozpiętość

Twierdzenie. Wartość optymalna modelu jest równa najmniejszej rozpiętości spośród wszystkich podziałów S na k grup.

Dowód. Ustalmy przypisanie x i odpowiadający mu podział P. Warunki dają $L \geq \max_j w(P_j)$ oraz $l \leq \min_j w(P_j)$, więc

$$L - l \geq \operatorname{rozp}(P).$$

Równość zachodzi dla $L = \max_j w(P_j)$ i $l = \min_j w(P_j)$. Te wartości są dopuszczalne, bo liczby w S są nieujemne. Dla ustalonego x najmniejsza wartość funkcji celu wynosi więc $\operatorname{rozp}(P)$. Minimalizacja po wszystkich x daje, z sekcji 3.2, minimum po wszystkich podziałach.

4. Jakość rozwiązania

4.1. Rozpiętość a OPT

Analizy w rozdziałach 2 i 3 oceniają wynik względem OPT, czyli najmniejszej możliwej sumy największej grupy. Model minimalizuje natomiast rozpiętość, czyli kryterium jakości przyjęte we wstępie (rozdział 1), którym benchmark w `src/testing.py` porównuje algorytmy. Niech $\Sigma = \sum_{i=1}^{n} a_i$.

Dla $k = 2$ kryteria są równoważne. Niech L i l oznaczają sumy większej i mniejszej grupy. Dwie grupy zawierają razem wszystkie liczby, więc $L + l = \Sigma$, skąd $l = \Sigma - L$ oraz $L - l = 2L - \Sigma$. Wartość Σ zależy tylko od danych wejściowych, a nie od podziału, więc rozpiętość rośnie wraz z L. Najmniejsza rozpiętość oznacza zatem najmniejszą sumę większej grupy.

Dla $k \geq 3$ kryteria mogą się różnić. Suma $\Sigma - L$ rozkłada się wtedy na co najmniej dwie grupy, więc Σ i L nie wyznaczają już l. Dla $S = \{8, 8, 8, 11, 13, 17\}$ i $k = 3$:

Podział	Sumy	Rozpiętość	Największa suma
$\{8, 11\}, \{8, 13\}, \{8, 17\}$	$19, 21, 25$	6	25
$\{8, 8, 8\}, \{11, 13\}, \{17\}$	$24, 24, 17$	7	24

Pierwszy podział ma najmniejszą rozpiętość, a drugi osiąga $OPT = 24$. Model wybierze pierwszy, więc nawet rozwiązanie optymalne dla modelu nie musi być optymalne w sensie OPT.

4.2. Gwarancja dla rozwiązania optymalnego modelu

Niech M będzie największą sumą grupy, a R rozpiętością podziału. Każda z pozostałych $k - 1$ grup ma sumę co najmniej $M - R$, więc

$$\Sigma \geq M + (k-1)(M - R), \qquad \text{czyli} \qquad M \leq \frac{\Sigma}{k} + \frac{k-1}{k} R.$$

Z oszacowania $OPT \geq \Sigma / k$ otrzymujemy

$$M - OPT \leq \frac{k-1}{k} R.$$

Rozwiązanie optymalne modelu ma rozpiętość nie większą niż jakikolwiek inny podział, w tym wynik algorytmu Karmarkara–Karpa, dla którego $R \leq a_{\max}$ (rozdział 2, sekcja 4.2). Dla podziału optymalnego w sensie modelu zachodzą więc te same gwarancje co dla tamtego algorytmu:

$$M - OPT \leq \frac{k-1}{k} a_{\max}, \qquad M \leq \left(2 - \frac{1}{k}\right) OPT.$$

W szczególności, gdy $R = 0$, wynik jest równy OPT.

4.3. Skutki limitu czasu

Gwarancje z sekcji 4.2 obowiązują tylko dla rozwiązania z udowodnioną optymalnością. Po upływie limitu możliwe są dwa przypadki:

Stan solvera	Status w PuLP	Zachowanie `algorithm_lp`
Znaleziono rozwiązanie, bez dowodu optymalności	`Optimal`	Zwraca najlepszy znaleziony podział, bez gwarancji jakości.

Stan solvera	Status w PuLP	Zachowanie `algorithm_lp`
Nie znaleziono rozwiązania	`Not Solved`	Zmienne nie mają wartości, więc kontrola kompletności zgłasza błąd `Solver returned an incomplete partition.`

Status `Optimal` nie oznacza więc, że wynik jest optymalny. Odróżnia to dopiero atrybut `sol_status` problemu, którego kod nie sprawdza. Na tej podstawie nie da się rozpoznać, czy zwrócony wynik ma gwarancje z sekcji 4.2.

Przerwanie jest częste, bo relaksacja modelu jest słaba. Przypisanie $x_{ij} = 1/k$ daje wszystkim grupom sumę Σ/k, więc $L = l$ i relaksacja ma wartość 0 niezależnie od danych. Do tego model nie usuwa symetrii: permutacja grup daje inne przypisanie x o tej samej wartości, więc każdy podział na niepuste grupy występuje w $k!$ wersjach.

W orkiestratorze błędy `algorithm_lp` są przechwytywane, a zamiast niego uruchamiany jest algorytm Karmarkara–Karpa.

5. Złożoność obliczeniowa

5.1. Założenia implementacyjne

- Model budujemy w PuLP 3.3.2.
- Solver HiGHS wywołujemy przez pakiet `highspy` z limitem `timeLimit=1`.
- Limit obejmuje tylko pracę solvera, a nie budowę modelu w Pythonie.
- Zakładamy $k \geq 1$. Dla $k = 0$ model jest sprzeczny i funkcja zgłasza błąd.

5.2. Analiza

Etap	Koszt
Utworzenie $nk + 2$ zmiennych	$O(nk)$
Budowa $n + 2k$ warunków ($3nk + 2k$ niezerowych współczynników)	$O(nk)$
Rozwiązanie bez limitu czasu, pesymistycznie	wykładniczy względem nk
Rozwiązanie z limitem czasu	ograniczony przez limit 1 s
Odczyt wartości zmiennych	$O(nk)$
Kontrola kompletności (sortowanie)	$O(n \log n)$

Możliwych przypisań liczb do grup jest k^n, a w najgorszym przypadku solver musi przejrzeć drzewo o rozmiarze wykładniczym, rozwiązując w węzłach relaksacje.

Złożoność czasowa: bez limitu wykładnicza. Z limitem wynosi $O(nk + n \log n)$ plus stały czas pracy solvera, ale kosztem utraty gwarancji optymalności.

Złożoność pamięciowa: $O(nk)$ na model oraz pamięć solvera na drzewo przeszukiwania.

5.3. Pomiary

Pojedyncze uruchomienia na liczbach losowanych jednostajnie z zakresu $1, \ldots, 1000$. Rozpiętość algorytmu Karmarkara–Karpa podano dla porównania.

n	k	Czas	Dowód optymalności	Rozpiętość LP	Rozpiętość KK
30	3	0,44 s	tak	0	2
100	5	1,05 s	nie	2	1
300	10	1,32 s	nie	503	1
1000	20	2,76 s	nie	1438	1

Dla małych instancji model daje wynik lepszy od algorytmów aproksymacyjnych. Dla dużych instancji jedna sekunda nie wystarcza i wynik jest wyraźnie gorszy. Czas ponad limit to budowa modelu i odczyt wyniku.

Rozdział 5

Pruning w modelu LP

1. Problem i zakres optymalizacji

Dla nieujemnych liczb całkowitych dzielimy elementy na `k` grup. Minimalizujemy rozpiętość ich sum:

$$D = \max_j S_j - \min_j S_j.$$

Co nie jest równoznaczne z minimalizacją największej sumy lub maksymalizacją minimalnej.[12]

2. Rozwiązanie początkowe i ograniczenie rozpiętości

Uruchamiamy KK i GNP na zredukowanym zbiorze, obliczamy rozpiętość każdego podziału i wybieramy mniejszy [14]. Jego wartość `U` jest górną granicą optimum. Do modelu dodajemy:

$$D \leq U.$$

Podczas branch-and-bound solver może odrzucać węzły, których poprawna dolna granica celu nie pozwala poprawić najlepszego znanego wyniku.

3. Granice sum grup wynikające z rozpiętości

Niech `T` oznacza sumę elementów `B`. Dla każdego podziału o rozpiętości nie większym niż `U` dodajemy:

$$\max\left(0, \left\lceil \frac{T-(k-1)U}{k} \right\rceil\right) \leq S_j \leq \left\lfloor \frac{T+(k-1)U}{k} \right\rfloor.$$

Niech `s` będzie najmniejszą sumą. Każda inna jest nie większa niż `s+U`, zatem

$$T \leq s + (k-1)(s+U) = ks + (k-1)U.$$

Analogicznie, dla największej sumy `M`, każda inna suma jest co najmniej równa `M-U`, więc

$$T \geq M + (k-1)(M-U) = kM - (k-1)U.$$

4. Redukcja symetrii przez porządek sum

Bezpośrednim odniesieniem dla tej metody w problemie multi-way number partitioning jest praca Michaela D. Moffitta [13], sekcja 4.4 „Pruning Symmetric Solutions”, s. 627 (piąta strona

PDF). Autor opisuje eliminowanie równoważnych numeracji grup przez wymaganie, aby sumy kolejnych podzbiorów były niemalejące. W jego notacji `S_j` oznacza podzbiór, więc warunek ma postać:

$$\sum_{a \in S_j} a \leq \sum_{a \in S_{j'}} a \qquad \text{dla } j < j'.$$

W naszym modelu symbol `S_j` oznacza już sumę grupy. Przyjmujemy odwrotną, równoważną konwencję numeracji — porządek nierosnący — i dodajemy `k-1` ograniczeń:

$$S_0 \geq S_1 \geq ... \geq S_{k-1}.$$

Grupy nie mają różnych pojemności ani innych indywidualnych własności. Można więc zmienić ich numerację bez zmiany celu.

5. Co gwarantuje obecna implementacja

- Pruning nie jest domyślnie włączany w samodzielnym wywołaniu LP dla całego wejścia.
- Nadal obowiązuje konfigurowalny limit czasu, domyślnie jedna sekunda.
- W najgorszym przypadku pozostaje kombinatoryczna trudność MILP. Pruning nie stanowi gwarancji wielomianowego czasu ani przyspieszenia każdej instancji.

Rozdział 6

Orkiestrator: działanie, złożoność i poprawność

Cel i oznaczenia

Wejście A zawiera n nieujemnych liczb całkowitych; $k \geq 1$ oznacza liczbę grup. Celem jest minimalizacja rozpiętości $D = \max(S_j) - \min(S_j)$, gdzie S_j to suma grupy. B oznacza wydzielony zbiór większych liczb, $m = |B|$, a R pozostałe wystąpienia. Dopuszczamy puste grupy. Orkiestrator jest rozwiązaniem heurystycznym: znajduje poprawny podział, ale nie gwarantuje minimalnego rozpiętości dla całego wejścia.

CV i wybór ścieżki

$\mathbf{CV = s/\bar{x}}$ opisuje zróżnicowanie danych względem średniej [15]. W kodzie $\bar{x}$ jest średnią, a s odchyleniem standardowym. Domyślny próg $\tau = 0{,}6$ jest parametrem projektu. Dla $CV \leq \tau$ orkiestrator dzieli całe wejście algorytmem KK. Dla $CV > \tau$ wybiera ścieżkę hybrydową: większe liczby dzieli LP, a pozostałe dokłada GNP. W dalszej części raportu nazywamy ją ścieżką *LP+GNP*.

Przebieg algorytmu

1. **Oblicz CV.** Jeśli $CV \leq \tau$, zwróć podział całego A uzyskany przez KK.
2. **Wydziel większe liczby.** Dla $CV > \tau$ utwórz B z wystąpień $a_i \geq \bar{x} + s$; pozostałe wystąpienia umieść w R.
3. **Podziel B przez LP.** Zastosuj model LP ze startem KK/GNP, ograniczeniem rozpiętości, granicami sum i nierosnącym porządkiem sum. Wszystkie te wartości oblicz wyłącznie dla B. Pusty B nie uruchamia solvera.
4. **Obsłuż błąd LP.** Jeśli wynik jest błędny lub solver nie zwróci rozwiązania dopuszczalnego, zapisz przyczynę i zastosuj KK do B.
5. **Dopełnij grupy przez GNP.** Przetwarzaj R malejąco, każdą liczbę dokładając do grupy o aktualnie najmniejszej sumie.
6. **Zweryfikuj wynik.** Sprawdź, czy powstało k grup i zachowano wszystkie wystąpienia liczb, łącznie z duplikatami.

Złożoność obliczeniowa

Poniższa analiza jest **analitycznym oszacowaniem kosztu obecnej implementacji**,.

- **Statystyki i podział:** obliczenie średniej, odchylenia, CV oraz wydzielenie B i R wymagają O(n) operacji.
- **GNP dla r elementów:** sortowanie kosztuje O(r log(r+1)), budowa kopca O(k), a przypisania O(r log(k+1)). Łącznie $\mathbf{G(r,k) = O(r\ log(r+1) + r\ log(k+1) + k)}$.
- **KK dla r elementów:** kopiec wymaga O(r log(r+1)), a sortowanie k grup przy scalaniu O(rk log(k+1)). Kopiowanie list elementów może kosztować do $O(r^2)$. Stąd zachowawcza granica $\mathbf{K(r,k) = O(r^2 + rk\ log(k+1) + r\ log(r+1) + k)}$.
- **LP dla B:** start KK/GNP kosztuje K(m,k) + G(m,k). Budowa modelu i przypisania startowego kosztuje O(mk+k); model ma mk zmiennych binarnych i O(m+k) ograniczeń. Czas samego solvera oznaczamy T_solve. Przestrzeń przypisań ma k^m elementów, lecz k^m nie jest dokładnym kosztem HiGHS: pruning ogranicza przeszukiwanie.
- **Dopełnienie i kontrola:** GNP dla R kosztuje G(n−m,k), a walidacja zachowania danych przez sortowanie O(n log(n+1) + k).

Ścieżka bez podziału kosztuje $\mathbf{O(n) + K(n,k)}$. Dla ścieżki hybrydowej otrzymujemy oszacowanie sumaryczne:

$$\mathbf{O(n\ log(n+1) + mk + k) + K(m,k) + G(m,k) + G(n-m,k) + T_solve.}$$

Rozdział 7

Architektura implementacji

Organizacja programu

Implementację opracowano w języku Python jako aplikację uruchamianą z wiersza poleceń. Zastosowano podział modułowy.

Podział odpowiedzialności

Moduł	Odpowiedzialność
`main.py`	Obsługa parametrów uruchomienia, wybór metody, inicjowanie eksperymentów i eksport wyników do CSV.
`orchestrator.py`	Analiza statystyczna danych, wydzielenie podproblemu, koordynacja algorytmów i połączenie ich wyników.
`algorithm_kk.py`	Realizacja heurystyki Karmarkara–Karpa z kopcem struktur częściowych podziałów.
`algorithm_gnp.py`	Zachłanne rozdzielanie liczb z użyciem kopca aktualnych sum grup.
`algorithm_lp.py`	Budowa modelu MILP, przygotowanie rozwiązania początkowego i ograniczeń pruningowych oraz obsługa solvera.
`testing.py`	Wczytywanie zestawów, walidacja podziałów, pomiar czasu oraz wyznaczanie i agregacja miar jakości.
`dataset_generator.py`	Generowanie i zapisywanie danych testowych z określonym ziarnem losowania, w tym zbiorów o zadanym współczynniku zmienności (CV) z wybranego rozkładu, oraz obliczanie CV zbioru.
`distributions.py`	Rozkłady używane przez generator (log-normalny, gamma, jednostajny) ze wspólnym interfejsem: losowanie wartości i dobór parametrów rozkładu dla zadanych średniej i CV.

Moduł	Odpowiedzialność
`experiments.py`	Wspólne funkcje eksperymentów: pomiar jakości i czasu działania algorytmu na zbiorze, przeglądy zakresu CV oraz siatki średniej i odchylenia standardowego.
`experiments_cv.py`	Wybór progu CV orkiestratora (rozdział 8): uruchomienie KK i ścieżki LP+GNP na zbiorach o różnym CV, krzywa regretu, wybór progu z walidacją krzyżową i przedziałem ufności bootstrap oraz próg minimax dla wielu scenariuszy.
`plots.py`	Wykresy wyników eksperymentów: mapy ciepła średniej i odchylenia standardowego oraz jakość i czas działania w zależności od CV.
`kk_vs_orchestrator.py`	Porównanie orkiestratora z algorytmem KK (rozdział 9): uruchomienie badania, zapis wyników do pliku JSON i tabele do raportu.

Reprezentacja danych i współpraca modułów

Dane wejściowe są reprezentowane jako lista nieujemnych liczb całkowitych. Wynik podziału ma postać listy grup, z których każda zawiera przypisane liczby. Powtarzające się wartości są zachowywane jako osobne wystąpienia, a grupy mogą pozostać puste.

KK i LP przyjmują zbiór liczb oraz żądaną liczbę grup. Interfejs GNP dodatkowo uwzględnia sumy istniejących grup i zwraca wyłącznie nowo przypisane elementy.

Moduł LP wykorzystuje bibliotekę PuLP do zapisania funkcji celu, zmiennych i ograniczeń, a następnie przekazuje model do solvera HiGHS. Rozwiązanie solvera jest przekształcane do list grup i sprawdzane przed zwróceniem.

Rozdział 8

Wybór progu CV orkiestratora

1. Problem

Orkiestrator (`src/orchestrator.py`) wybiera ścieżkę na podstawie współczynnika zmienności $\mathrm{CV} = s/\bar{x}$:

- $\mathrm{CV} \leq \tau$ – cały zbiór dzieli KK,
- $\mathrm{CV} > \tau$ – duże liczby dzieli LP, a resztę dokłada GNP (ścieżka LP+GNP).

Domyślny próg $\tau = 0{,}6$ został ustawiony arbitralnie. Ten dokument opisuje metodę empirycznego wyboru τ, zaimplementowaną w `src/experiments_cv.py`.

2. Kluczowa obserwacja

Próg nie zmienia działania żadnej z gałęzi, decyduje tylko, która z nich zostanie użyta. Wystarczy więc dla każdej instancji x raz obliczyć wynik obu gałęzi:

- $c_{KK}(x)$ – koszt podziału KK (to samo co `cv_threshold = inf`),
- $c_{LP}(x)$ – koszt ścieżki LP+GNP, wymuszonej przez `cv_threshold = 0`.

Koszt orkiestratora dla dowolnego progu τ wynosi wtedy

$$c_\tau(x) = \begin{cases} c_{KK}(x) & \text{gdy } \mathrm{CV}(x) \leq \tau, \\ c_{LP}(x) & \text{w przeciwnym razie.} \end{cases}$$

Przegląd dowolnej liczby progów nie wymaga ponownego uruchamiania algorytmów.

3. Funkcja celu

Surowe $D = \max_j S_j - \min_j S_j$ rośnie razem z CV, więc nie nadaje się do uśredniania po różnych instancjach. Domyślnym kosztem jest rozpiętość znormalizowany sumą, $D/\sum_i a_i$ (pole `std_max`). Opcjonalnie można doliczyć czas:

$$c(x) = \frac{D(x)}{\sum_i a_i} + \lambda \cdot t(x),$$

gdzie λ to parametr `time_weight`.

Progi porównujemy przez średni **regret**, czyli stratę względem wyroczni, która zawsze wybiera lepszą gałąź:

$$\mathrm{regret}(\tau) = \frac{1}{N} \sum_x \Big(c_\tau(x) - \min\left(c_{KK}(x),\, c_{LP}(x)\right) \Big).$$

4. Procedura

1. **Zbieranie danych** (`collect_branch_results`). Dla zadanych wartości CV, rozkładu, n i k generujemy zbiory i uruchamiamy na nich obie gałęzie. Zapisujemy zmierzone CV oraz D, $D/\sum$ i czas każdej gałęzi.
2. **Kandydaci na próg** (`candidate_thresholds`). Funkcja regretu zmienia się tylko między kolejnymi zmierzonymi wartościami CV. Wystarczy więc sprawdzić 0 (zawsze LP+GNP), środki między kolejnymi CV oraz największe CV (zawsze KK).
3. **Krzywa regretu** (`regret_curve`). Po posortowaniu instancji po CV koszt dla każdego progu liczymy z sum prefiksowych w czasie $O(N \log N)$.
4. **Wybór progu** (`best_threshold`). Jeśli minimum osiąga kilka progów, wybieramy środkowy z nich, a nie skrajny, bo lepiej uogólnia się na nowe dane.
5. **Walidacja krzyżowa** (`cross_validate_threshold`). Próg dobieramy na $k-1$ częściach danych i mierzymy jego regret na części odłożonej. Dla porównania liczymy też regret progu 0,6, strategii „zawsze KK" i „zawsze LP+GNP".
6. **Niepewność** (`bootstrap_threshold`). Losowanie ze zwracaniem daje 95% przedział ufności dla τ^* i pasmo ufności dla całej krzywej regretu.
7. **Wiele scenariuszy** (`minimax_threshold`). Gdy τ^* różni się między rozkładami lub wartościami k, wybieramy próg minimalizujący najgorszy średni regret spośród scenariuszy.

Jeśli optymalny próg wyraźnie zależy od n lub k, to samo CV nie wystarcza do wyboru gałęzi. W takim przypadku można zamiast progu dopasować mały klasyfikator (np. płytkie drzewo decyzyjne) na cechach CV, n, k.

5. Uruchomienie

```
cd src && uv run python experiments_cv.py
```

Domyślnie jest to 30 wartości CV $\times$ 10 zbiorów dla każdego rozkładu, a jedno wywołanie LP może trwać do `time_limit = 1` s. Pełne badanie zajmuje około dziesięć minut:

```
from experiments_cv import run_threshold_study, print_threshold_study
study = run_threshold_study(ks=(3, 5), n=100, num_datasets=10, time_limit=1, seed=0)
print_threshold_study(study)
```

6. Wyniki

Badanie na większych danych: rozkłady lognormal, gamma i uniform, $k \in \{3, 5\}$, $n = 100$, 10 zbiorów na wartość CV, `time_limit = 1` s, koszt $D/\sum$, `seed = 0`:

```
run_threshold_study(ks=(3, 5), n=100, num_datasets=10, time_limit=1, seed=0)
```

Scenariusz	τ^*	95% CI	regret (test)	$\tau = 0{,}6$	zawsze KK	zawsze LP+GNP
lognormal, k=3	4,50	[3,93; 4,59]	3,3e-06	2,8e-04	3,3e-06	7,3e-04
gamma, k=3	2,73	[2,30; 3,86]	5,0e-07	7,5e-05	4,8e-07	4,7e-04
uniform, k=3	0,60	[0,56; 0,60]	3,5e-05	≈0	≈0	2,4e-03
lognormal, k=5	4,50	[3,39; 4,59]	0	4,5e-04	≈0	8,5e-04

Scenariusz	τ^*	95% CI	regret (test)	$\tau = 0{,}6$	zawsze KK	zawsze LP+GNP
gamma, k=5	2,67	[2,41; 2,87]	2,9e-07	9,7e-05	2,9e-07	3,6e-04
uniform, k=5	0,60	[0,56; 0,60]	1,2e-05	≈0	≈0	2,7e-04
łącznie	4,26	[3,86; 4,26]	9,4e-07	2,1e-04	9,4e-07	6,6e-04

Próg minimax wynosi 4,26 (najgorszy średni regret $3{,}3 \cdot 10^{-6}$).

- Dla lognormal i gamma τ^* leży na największym zmierzonym CV lub blisko niego, więc w praktyce wybierana jest strategia „zawsze KK". Próg 0,6 ma regret o 2–3 rzędy wielkości większy, a „zawsze LP+GNP" jest jeszcze gorsza.
- Dla uniform wychodzi $\tau^* = 0{,}6$, ale „zawsze KK" ma regret ≈0. Zakres CV tego rozkładu jest wąski (po 50 zbiorów na scenariusz), więc krzywa regretu jest płaska i wybór progu jest tam mało wiarygodny.
- Różnice między $k = 3$ i $k = 5$ są małe, a przedziały ufności τ^* się pokrywają. Nie widać wyraźnej zależności progu od k.

Rozdział 9

Porównanie orkiestratora z algorytmem Karmarkara–Karpa

1. Cel

Orkiestrator (`src/orchestrator.py`) przy CV $\leq \tau$ dzieli cały zbiór algorytmem KK, a przy CV $> \tau$ wybiera ścieżkę LP+GNP opisaną w rozdziale 6. Celem badania jest sprawdzenie, czy orkiestrator daje lepsze podziały niż sam algorytm KK i jakim kosztem czasowym. Porównujemy dwa progi:

- $\tau = 0{,}6$ – próg domyślny,
- $\tau = 4{,}26$ – próg minimax wyznaczony w rozdziale o wyborze progu CV.

2. Metoda

Podobnie jak przy wyborze progu, korzystamy z tego, że próg decyduje tylko o wyborze ścieżki. Każdy zbiór dzielimy raz algorytmem KK i raz orkiestratorem z progiem 0, który na każdym zbiorze używa LP+GNP. Wynik orkiestratora z progiem τ to wynik KK, gdy zmierzone CV zbioru nie przekracza τ, a w przeciwnym razie wynik LP+GNP. Oba progi oceniamy więc na tych samych zbiorach.

Dane wygenerowano generatorem o zadanym CV:

- rozkłady log-normalny, gamma i jednostajny, średnia 10 000,
- $n \in \{100, 1000\}$, $k \in \{3, 5\}$,
- zadane CV: $0{,}1; 0{,}2; \ldots; 0{,}5$ oraz $0{,}6; 0{,}8; \ldots; 6{,}0$ (wartości nieosiągalne dla rozkładu są pomijane, dla rozkładu jednostajnego zostaje 0,1–0,5),
- 5 zbiorów na każdą kombinację (rozkład, n, k, CV), łącznie 1420 zbiorów,
- limit czasu LP: 1 s, `seed = 0`.

Generator każdego scenariusza (rozkład, n, k) ma ziarno wyprowadzone z `seed`, więc zmiana ustawień jednego scenariusza nie zmienia zbiorów pozostałych.

Miarą jakości jest rozpiętość $D = \max_j S_j - \min_j S_j$. Oznaczamy przez D_{KK} rozpiętość podziału KK, przez D_{LP} rozpiętość podziału LP+GNP, a przez D_{O} rozpiętość podziału orkiestratora z danym progiem. Wyniki porównujemy na każdym zbiorze osobno oraz za pomocą ilorazu

$$q = \frac{D_{\mathrm{LP}} + 1}{D_{\mathrm{KK}} + 1}.$$

Wartość $q > 1$ oznacza, że KK dał mniejszą rozpiętość, a jedynka w liczniku i mianowniku pozwala porównywać podziały o rozpiętości 0. Czas mierzymy dla każdego zbioru i podajemy jego medianę oraz 95. percentyl.

3. Uruchomienie

```
cd src
uv run python kk_vs_orchestrator.py --quick        # szybki test
uv run python kk_vs_orchestrator.py --seed 0       # pełne badanie
uv run python kk_vs_orchestrator.py --report-only  # tylko raport
```

Wyniki trafiają do `src/results/kk_vs_orchestrator/`: `results.json` (wszystkie zbiory), `report.md` (tabele). Pełne badanie trwało około 16 minut (Python 3.13, PuLP 4.0.0, highspy 1.15.1, Windows 11, procesor AMD).

4. Wyniki

4.1. Zmierzone CV

Rozkład	n	mediana CV	max CV	CV > 0,6	CV > 4,26	udział jedynek
lognormal	100	1,71	7,06	83%	2%	0%
lognormal	1000	2,34	17,70	83%	8%	0%
gamma	100	2,57	7,61	83%	15%	29,8%
gamma	1000	2,71	7,70	83%	24%	29,6%
uniform	100	0,31	0,56	0%	0%	0%
uniform	1000	0,30	0,52	0%	0%	0%

Dla rozkładu log-normalnego zmierzone CV jest przy dużym zadanym CV wyraźnie niższe od zadanego: dla zadanego CV = 5 średnio 2,61 przy $n = 100$ i 3,81 przy $n = 1000$. Rozkład gamma odtwarza zadane CV dokładniej (4,04 i 4,72), ale jedynki stanowią średnio około 30% wartości wszystkich zbiorów z tego rozkładu, a przy dużym CV jeszcze więcej. Rozkład jednostajny nie przekracza progu 0,6, więc orkiestrator zawsze używa na nim KK.

4.2. Orkiestrator a KK

Każdy scenariusz obejmuje 165 zbiorów. Kolumna „CV > τ" podaje liczbę zbiorów, na których orkiestrator wybrał LP+GNP, a mediana q jest liczona tylko na tych zbiorach. Na 100 zbiorach z rozkładu jednostajnego orkiestrator przy obu progach daje ten sam wynik co KK, dlatego tabele ich nie obejmują.

Próg $\tau = 0{,}6$:

Scenariusz	CV > τ	$D_O < D_{KK}$	$D_O = D_{KK}$	$D_O > D_{KK}$	mediana q
lognormal, $n = 100$, $k = 3$	136	0	39	126	28,9
lognormal, $n = 100$, $k = 5$	137	5	55	105	3,6
lognormal, $n = 1000$, $k = 3$	137	0	34	131	11,0
lognormal, $n = 1000$, $k = 5$	137	0	29	136	14,0
gamma, $n = 100$, $k = 3$	136	1	132	32	1,0
gamma, $n = 100$, $k = 5$	137	8	115	42	1,0
gamma, $n = 1000$, $k = 3$	136	0	152	13	1,0
gamma, $n = 1000$, $k = 5$	138	0	149	16	1,0

Próg $\tau = 4{,}26$:

Scenariusz	CV $> \tau$	$D_{\mathrm{O}} < D_{\mathrm{KK}}$	$D_{\mathrm{O}} = D_{\mathrm{KK}}$	$D_{\mathrm{O}} > D_{\mathrm{KK}}$	mediana q
lognormal, $n = 100$, $k = 3$	6	0	165	0	1,0
lognormal, $n = 100$, $k = 5$	2	0	164	1	1,0
lognormal, $n = 1000$, $k = 3$	15	0	153	12	3,0
lognormal, $n = 1000$, $k = 5$	13	0	153	12	6,0
gamma, $n = 100$, $k = 3$	26	0	164	1	1,0
gamma, $n = 100$, $k = 5$	25	0	163	2	1,0
gamma, $n = 1000$, $k = 3$	38	0	165	0	1,0
gamma, $n = 1000$, $k = 5$	42	0	165	0	1,0

Łącznie na 1420 zbiorach:

- przy progu 0,6 orkiestrator wybrał LP+GNP na 1094 zbiorach (77%). Dał mniejszą rozpiętość niż KK na 14 z nich, równą na 479 i większą na 601,
- przy progu 4,26 wybrał LP+GNP na 167 zbiorach (12%). Na żadnym z nich nie dał mniejszej rozpiętości niż KK, na 139 dał równą, a na 28 większą.

4.3. LP+GNP a KK w przedziałach zmierzonego CV

Tabele porównują LP+GNP i KK na wszystkich zbiorach, niezależnie od progu.

Rozkład log-normalny:

n	CV	$D_{\mathrm{LP}} < D_{\mathrm{KK}}$	$D_{\mathrm{LP}} = D_{\mathrm{KK}}$	$D_{\mathrm{LP}} > D_{\mathrm{KK}}$	mediana D_{KK}	mediana D_{LP}	mediana q
100	(0; 0,6]	0	0	57	8	1915	128,8
100	(0,6; 1]	0	0	22	6	834	98,2
100	(1; 2]	2	0	118	4	124	15,7
100	(2; 3]	3	17	77	6	56	6,1
100	(3; 4,26]	0	13	13	47015	47015	1,0
100	$>$ 4,26	0	7	1	303313	320326	1,0
1000	(0; 0,6]	0	0	56	1	2310	1052,8
1000	(0,6; 1]	0	0	21	1	611	375,5
1000	(1; 2]	0	0	60	1	134	69,0
1000	(2; 3]	0	0	97	1	15	10,5
1000	(3; 4,26]	0	3	65	1	9	6,8
1000	$>$ 4,26	0	4	24	1	8	4,5

Rozkład gamma:

n	CV	$D_{\mathrm{LP}} < D_{\mathrm{KK}}$	$D_{\mathrm{LP}} = D_{\mathrm{KK}}$	$D_{\mathrm{LP}} > D_{\mathrm{KK}}$	mediana D_{KK}	mediana D_{LP}	mediana q
100	(0; 0,6]	0	0	57	8	1779	144,8
100	(0,6; 1]	0	0	20	20	366	18,1
100	(1; 2]	4	18	37	1	8	2,2
100	(2; 3]	1	42	8	1	1	1,0
100	(3; 4,26]	4	82	6	824	1110	1,0
100	$>$ 4,26	0	48	3	219305	219305	1,0
1000	(0; 0,6]	2	0	54	1	1108	503,2
1000	(0,6; 1]	0	0	18	1	124	79,0

n	CV	$D_{\mathrm{LP}} < D_{\mathrm{KK}}$	$D_{\mathrm{LP}} = D_{\mathrm{KK}}$	$D_{\mathrm{LP}} > D_{\mathrm{KK}}$	mediana D_{KK}	mediana D_{LP}	mediana q
1000	(1; 2]	0	44	11	1	1	1,0
1000	(2; 3]	0	52	0	1	1	1,0
1000	(3; 4,26]	0	69	0	1	1	1,0
1000	$> 4{,}26$	0	80	0	1	1	1,0

Rozkład jednostajny:

n	CV	$D_{\mathrm{LP}} < D_{\mathrm{KK}}$	$D_{\mathrm{LP}} = D_{\mathrm{KK}}$	$D_{\mathrm{LP}} > D_{\mathrm{KK}}$	mediana D_{KK}	mediana D_{LP}	mediana q
100	(0; 0,6]	1	0	49	12	541	13,8
1000	(0; 0,6]	2	0	48	1	200	15,8

Łącznie LP+GNP dało mniejszą rozpiętość niż KK na 19 zbiorach, równą na 479 i większą na 922. Mediana q wyniosła 6,0.

- Przy małym CV LP+GNP daje większą rozpiętość niż KK na prawie wszystkich zbiorach, a mediana q sięga od kilkunastu do ponad tysiąca. Przewaga KK maleje wraz ze wzrostem CV.
- Równe rozpiętości występują w dwóch sytuacjach. Przy $n = 1000$ dla rozkładu gamma oba podejścia osiągają $D = 1$. Przy bardzo dużym CV i $n = 100$ oba dają tę samą, bardzo dużą rozpiętość (mediany rzędu 10^4–10^5), porównywalną ze średnią sumą podzbioru $\Sigma/k \approx 2$–$3{,}3 \cdot 10^5$. Wskazuje to, że rozpiętość wyznacza wtedy jedna lub kilka dominujących liczb, których żaden podział nie zrównoważy.
- Dla rozkładu log-normalnego przy $n = 1000$ LP+GNP nie dało mniejszej rozpiętości niż KK na żadnym zbiorze, a KK osiągał medianę $D = 1$ w każdym przedziale CV.

4.4. Wpływ k

Dane	$D_{\mathrm{LP}} < D_{\mathrm{KK}}$	$D_{\mathrm{LP}} = D_{\mathrm{KK}}$	$D_{\mathrm{LP}} > D_{\mathrm{KK}}$	mediana q
lognormal, $k = 3$	0	16	314	24,5
lognormal, $k = 5$	5	28	297	11,7
gamma, $k = 3$	3	226	101	1,0
gamma, $k = 5$	8	209	113	1,0
uniform, $k = 3$	3	0	47	34,0
uniform, $k = 5$	0	0	50	13,9

Dla $k = 5$ przewaga KK jest mniejsza niż dla $k = 3$ (dla rozkładu log-normalnego mediana q spada z 24,5 do 11,7), ale wnioski pozostają te same.

4.5. Czas działania

Dane	KK, mediana [ms]	KK, p95 [ms]	LP+GNP, mediana [ms]	LP+GNP, p95 [ms]	LP przerwane limitem czasu
$n = 100$, $k = 3$	0,30	0,44	60,69	1005,83	46 z 355
$n = 100$, $k = 5$	0,39	0,62	192,43	1010,92	107 z 355
$n = 1000$, $k = 3$	3,53	5,76	1016,64	1043,17	339 z 355
$n = 1000$, $k = 5$	4,64	14,21	1026,94	1075,69	350 z 355

LP+GNP działa o dwa–trzy rzędy wielkości dłużej niż KK. Przy $n = 1000$ do LP trafia średnio od około 20 do ponad 200 liczb i solver prawie zawsze kończy pracę po upływie limitu czasu, bez dowodu optymalności. Na 14 zbiorach rozwiązanie początkowe dla dużych liczb (lepszy z podziałów KK i GNP) miało rozpiętość 0, więc LP nie było uruchamiane (status `HeuristicOptimal`). Na żadnym zbiorze orkiestrator nie przeszedł awaryjnie do KK.

4.6. Zbiory, na których LP+GNP daje mniejszą rozpiętość

Największą przewagę LP+GNP uzyskało na dwóch zbiorach z rozkładu gamma przy $n = 100$ i CV 3,56 oraz 3,04: KK dał $D = 479$ i $D = 457$, a LP+GNP $D = 1$. Pozostałe przypadki przewagi są mniejsze. Pięć z 19 takich zbiorów ma CV $\leq 0{,}6$, więc orkiestrator i tak wybiera na nich KK.

5. Podsumowanie

- Algorytm KK jest bardzo silnym punktem odniesienia: LP+GNP dało mniejszą rozpiętość tylko na 19 z 1420 zbiorów. Przy małym i średnim CV rozpiętość LP+GNP jest wyraźnie większa, a przy dużym CV oba podejścia dają najczęściej ten sam wynik.
- Próg 0,6 przełącza orkiestrator zbyt często: na 42% zbiorów daje on większą rozpiętość niż KK.
- Przy progu 4,26 orkiestrator zachowuje jakość KK na 98% zbiorów; większą rozpiętość daje na 28 zbiorach, głównie z rozkładu log-normalnego przy $n = 1000$.
- Wybór LP+GNP wydłuża czas działania do około 1 s przy $n = 1000$, wobec kilku milisekund dla KK.

Rozdział 10

Wnioski

1. Zakres zrealizowanych prac

W ramach projektu zaimplementowano trzy metody podziału zbioru liczb naturalnych na k podzbiorów: algorytm Karmarkara–Karpa (KK), algorytm zachłanny (GNP) oraz model programowania liniowego całkowitoliczbowego (LP), rozwiązywany solverem HiGHS z limitem czasu. Model LP rozszerzono o techniki ograniczania przestrzeni przeszukiwania (pruning): rozwiązanie początkowe wyznaczane przez KK i GNP, ograniczenie rozpiętości, granice sum podzbiorów oraz eliminację symetrii.

Na podstawie tych metod zbudowano główny moduł projektu, orkiestrator, który w zależności od współczynnika zmienności danych (CV) stosuje algorytm KK albo ścieżkę LP+GNP, w której liczby duże rozdziela model LP, a pozostałe algorytm GNP. Projekt obejmuje ponadto generator zbiorów danych o zadanym CV (rozkłady log-normalny, gamma i jednostajny) oraz narzędzia do porównywania algorytmów i doboru progu CV.

2. Najważniejsze wyniki

2.1. Gwarancje teoretyczne

Dla algorytmów KK i GNP wyprowadzono wspólne oszacowanie $M \leq \frac{\Sigma}{k} + \frac{k-1}{k}\, a_{\max}$, z którego wynika gwarancja aproksymacji $M \leq \left(2 - \frac{1}{k}\right) OPT$. W literaturze dla obu algorytmów znane jest silniejsze ograniczenie $\left(\frac{4}{3} - \frac{1}{3k}\right) OPT$: dla algorytmu zachłannego z sortowaniem malejącym [6], a dla algorytmu Karmarkara–Karpa [3]. Gwarancje najgorszego przypadku obu metod są zatem praktycznie takie same i nie wskazują, która z nich daje lepsze podziały; różnice między nimi ujawniają się dopiero w badaniach empirycznych.

Model LP jest dokładny w tym sensie, że jego rozwiązanie optymalne minimalizuje rozpiętość podziału. Dla $k \geq 3$ podział o minimalnej rozpiętości nie musi jednak minimalizować sumy największego podzbioru, czyli wartości OPT (rozdział 4, sekcja 4.1). Jeżeli obliczenia zostaną przerwane po przekroczeniu limitu czasu, wynik LP nie ma żadnej gwarancji jakości.

2.2. Zachowanie w praktyce

Pomiary przedstawione w rozdziale 4 wskazują, że model LP jest konkurencyjny wyłącznie dla małych instancji. Dla $n = 30$ i $k = 3$ solver wykazał optymalność podziału o rozpiętości 0, podczas gdy KK uzyskał rozpiętość 2. Począwszy od $n = 100$, limit jednej sekundy nie wystarcza do wykazania optymalności, a dla $n = 1000$ i $k = 20$ rozpiętość uzyskana przez LP wyniosła 1438, wobec 1 dla KK. Na danych losowych algorytm Karmarkara–Karpa osiąga rozpiętości wielokrotnie mniejsze od pojedynczych elementów zbioru wejściowego.

2.3. Dane testowe a próg orkiestratora

Pierwotne zbiory testowe generowano z rozkładu jednostajnego, dla którego CV $\leq 1/\sqrt{3} \approx 0{,}577$, więc orkiestrator nigdy nie wybierał na nich ścieżki LP+GNP. Dlatego opracowano generator zbiorów o zadanym CV, oparty na rozkładach log-normalnym i gamma, z rozkładem jednostajnym jako punktem odniesienia. CV zmierzone na próbie bywa istotnie niższe od zadanego (dla rozkładu log-normalnego i zadanego CV $= 5$ średnio 2,61 przy $n = 100$), dlatego w eksperymentach posługujemy się CV **zmierzonym** dla każdego zbioru, ponieważ to ono stanowi podstawę decyzji orkiestratora.

2.4. Orkiestrator

Badanie progu CV (rozdział 8) wskazało progi bliskie największemu zmierzonemu CV: 4,50 dla rozkładu log-normalnego, około 2,7 dla rozkładu gamma i 4,26 jako próg minimax. Regret progu domyślnego 0,6 był o dwa–trzy rzędy wielkości większy. Porównanie na 1420 zbiorach (rozdział 9) potwierdziło ten wynik dla $n \in \{100, 1000\}$ i $k \in \{3, 5\}$: przy progu 0,6 orkiestrator dał większą rozpiętość niż KK na 42% zbiorów, a przy progu 4,26 zachował jakość KK na 98% zbiorów.

Przy $n = 100$ model LP w większości przypadków kończy obliczenia z dowodem optymalności, a mimo to LP+GNP daje większą rozpiętość niż KK. Słabym punktem ścieżki LP+GNP jest więc nie podział liczb dużych, lecz rozdzielanie pozostałych liczb algorytmem GNP, który robi to mniej równomiernie niż KK.

Dane nie potwierdziły hipotezy, że wysokie CV wskazuje zbiory korzystne dla odrębnego podziału liczb dużych i małych. Wyniki dają jednak jasne zalecenie: przy wysokim progu orkiestrator działa praktycznie jak KK i nie traci na jakości, a dalsze prace nad ścieżką LP+GNP powinny dotyczyć sposobu rozdzielania małych liczb (sekcja 4, punkt 1).

3. Ograniczenia

- Wszystkie eksperymenty przeprowadzono na danych syntetycznych pochodzących z trzech rodzin rozkładów.
- Orkiestrator badano dla $n \in \{100, 1000\}$ i $k \in \{3, 5\}$; zachowanie dla większych wartości nie zostało sprawdzone.
- Wyniki modelu LP zależą od przyjętego limitu czasu (1 s) i od sprzętu, a pomiary czasu nie są w pełni powtarzalne.

4. Perspektywy i dalsze prace

1. Modyfikacja ścieżki LP+GNP przez zastąpienie algorytmu GNP metodą różnicową w stylu KK, uwzględniającą sumy podzbiorów wyznaczone dla liczb dużych.
2. Zastosowanie przy wyborze algorytmu bogatszego zestawu cech niż sama wartość CV, np. stosunku największego elementu do Σ/k, stosunku n/k lub udziału małych liczb, oraz prostego klasyfikatora (np. płytkiego drzewa decyzyjnego) w miejsce pojedynczego progu.
3. Wykorzystanie dla małych instancji metod dokładnych, np. pełnego algorytmu Karmarkara–Karpa (CKK) [16] lub strategii przeszukiwania opisanych przez Moffitta [13], jako alternatywy dla modelu MILP z limitem czasu.
4. Weryfikacja wniosków na danych rzeczywistych (np. czasach wykonania zadań lub rozmiarach plików) oraz na rozkładach o cięższych ogonach.

Bibliografia

[1] N. Karmarkar, R. M. Karp, *The differencing method of set partitioning*, Technical Report UCB/CSD 82/113, Computer Science Division, University of California, Berkeley, 1982.

[2] M. Fischetti, S. Martello, *Worst-case analysis of the differencing method for the partition problem*, Mathematical Programming 37(1), 1987, s. 117–120, doi:10.1007/BF02591687.

[3] W. Michiels, J. Korst, E. Aarts, J. van Leeuwen, *Performance ratios of the Karmarkar–Karp differencing method*, Journal of Combinatorial Optimization 13(1), 2007, s. 19–32, doi:10.1007/s10878-006-9010-z.

[4] *Largest differencing method*, Wikipedia, https://en.wikipedia.org/wiki/Largest_differencing_method.

[5] R. L. Graham, *Bounds for certain multiprocessing anomalies*, The Bell System Technical Journal 45(9), 1966, s. 1563–1581, doi:10.1002/j.1538-7305.1966.tb01709.x.

[6] R. L. Graham, *Bounds on multiprocessing timing anomalies*, SIAM Journal on Applied Mathematics 17(2), 1969, s. 416–429, doi:10.1137/0117039.

[7] *Greedy number partitioning*, Wikipedia, https://en.wikipedia.org/wiki/Greedy_number_partitioning.

[8] A. H. Land, A. G. Doig, *An automatic method of solving discrete programming problems*, Econometrica 28(3), 1960, s. 497–520, doi:10.2307/1910129.

[9] Q. Huangfu, J. A. J. Hall, *Parallelizing the dual revised simplex method*, Mathematical Programming Computation 10(1), 2018, s. 119–142, doi:10.1007/s12532-017-0130-5.

[10] S. Mitchell, M. O'Sullivan, I. Dunning, *PuLP: A Linear Programming Toolkit for Python*, The University of Auckland, 2011.

[11] *Multiway number partitioning*, Wikipedia, https://en.wikipedia.org/wiki/Multiway_number_partitioning.

[12] Richard E. Korf (2010). *Objective Functions for Multi-Way Number Partitioning*. Proceedings of the International Symposium on Combinatorial Search, 1(1), 71–72. Strona publikacji i dostęp do PDF. DOI: 10.1609/socs.v1i1.18172. Powołanie: nierównoważność trzech funkcji celu dla co najmniej trzech grup.

[13] Michael D. Moffitt (2013). *Search Strategies for Optimal Multi-Way Number Partitioning*. Proceedings of the Twenty-Third International Joint Conference on Artificial Intelligence (IJCAI 2013), 623–629. Pełny tekst PDF · Sekcja 4.4 „Pruning Symmetric Solutions", s. 627.

[14] Dr Rafał Górak. *Konsultacja osobista*. Zakres powołania: uzgodnienie pomysłu wprowadzenia rozwiązania początkowego dla modelu LP i wykonania pruningu.

[15] George F. Reed, Freyja Lynn, Bruce D. Meade (2002). *Use of Coefficient of Variation in Assessing Variability of Quantitative Assays*. Clinical and Diagnostic Laboratory Immunology, 9(6), 1235–1239. Pełny tekst · DOI.

[16] R. E. Korf, *A complete anytime algorithm for number partitioning*, Artificial Intelligence 106(2), 1998, s. 181–203, doi:10.1016/S0004-3702(98)00086-1.